## 1. INTRODUCCIÓN Y CONTEXTO

Este taller evaluativo corresponde al cierre del trimestre de la competencia de Desarrollo de Software con Frameworks del programa Análisis y Desarrollo de Software (228185). Durante el trimestre los aprendices trabajaron con Node.js, React, Django, Laravel, consumo y construcción de APIs REST, y bases de datos relacionales con PostgreSQL y MySQL.

El objetivo es que cada grupo diseñe, desarrolle y despliegue una solución de software completa a un problema cotidiano real, eligiendo libremente el stack tecnológico visto durante la formación. El producto final debe estar funcional, desplegado y documentado.

## 2. ORGANIZACIÓN DE GRUPOS Y ROLES

Los 29 aprendices se distribuyen en 9 grupos de 3 integrantes y 1 grupo de 2 integrantes. Cada grupo tiene roles definidos con responsabilidades específicas que serán evaluadas individualmente durante la sustentación.

| Rol | Responsable principal | Responsabilidades |
| --- | --- | --- |
| **Backend Developer** | Integrante 1 | Lógica del servidor, base de datos, endpoints de la API, autenticación y manejo de errores |
| **Frontend Developer** | Integrante 2 | Interfaz de usuario, consumo de la API, experiencia de usuario y diseño responsivo |
| **Git Master + Documentador** | Integrante 3 | Gestión del repositorio, ramas, Pull Requests, documentación técnica completa y coordinación del despliegue |

Nota para el grupo de 2: el integrante 2 asume el rol de Frontend Developer + Git Master + Documentador.

## 3. REQUISITOS MÍNIMOS DEL PROYECTO

Independientemente del problema asignado, todos los proyectos deben cumplir los siguientes requisitos técnicos obligatorios:

- Mínimo 5 tablas relacionadas en la base de datos con integridad referencial
- Autenticación de usuarios con al menos 2 roles diferenciados (ej: administrador y usuario común)
- Al menos 1 endpoint o vista de reporte que agregue datos (no solo operaciones CRUD básicas)

- Sistema desplegado y accesible: puede ser en un servicio gratuito como Railway, Render, Vercel o similar
- Repositorio en GitHub/GitLab con historial de commits visible por integrante
- Mínimo 10 commits por integrante con mensajes descriptivos en español
- Al menos 1 rama de desarrollo diferente a main y 1 Pull Request documentado
- README con instrucciones claras de instalación, configuración y despliegue
- Stack tecnológico declarado desde el inicio en el README

## 4. STACK TECNOLÓGICO PERMITIDO

Cada grupo elige libremente su stack entre las tecnologías vistas durante el trimestre. La elección debe quedar registrada en el README del repositorio desde la primera sesión de trabajo.

| Backend | Frontend | Base de datos | Despliegue sugerido |
|---|---|---|---|
| Django, Laravel o Node.js + Express | React, HTML/CSS/JS vanilla o cualquier template engine | PostgreSQL o MySQL | Railway, Render, Vercel, Heroku, VPS propio |

## 5. PROBLEMAS ASIGNADOS POR GRUPO

Cada grupo recibe un problema diferente. El problema es la guía, pero el grupo tiene libertad de tomar decisiones de diseño siempre que los requisitos mínimos y los retos técnicos estén cubiertos.

### 🏘 GRUPO 1 — SISTEMA DE PRÉSTAMOS ENTRE VECINOS

**PROBLEMA: Cristian, castaño, bonilla**

App para conjuntos residenciales donde los vecinos pueden publicar objetos disponibles para préstamo (herramientas, electrodomésticos, implementos deportivos), otros vecinos los solicitan, y el sistema gestiona todo el ciclo del préstamo desde la solicitud hasta la devolución.

**RETOS TÉCNICOS:**

- Estados del préstamo: disponible → solicitado → en préstamo → devuelto → retrasado
- Lógica de alertas automáticas por fecha de devolución vencida
- Sistema de calificaciones bidireccional: quien presta califica a quien pide y viceversa
- Historial completo por objeto y por usuario con trazabilidad total

### 💸 GRUPO 2 — SEGUIMIENTO DE DEUDAS ENTRE AMIGOS (SPLITWISE COLOMBIANO)

**PROBLEMA: harold, villamil, edwin**

Grupos de amigos registran gastos compartidos — la salida, el mercado, el regalo de cumpleaños — y el sistema calcula automáticamente quién le debe a quién, optimizando los pagos para minimizar el número de transferencias necesarias. Todo en pesos colombianos (COP).

**RETOS TÉCNICOS:**

- Algoritmo de liquidación de deudas: no es solo dividir, es optimizar quién paga a quién para saldar con el mínimo de transacciones
- Manejo de grupos con miembros que entran y salen en distintos momentos
- Historial de pagos parciales y saldos pendientes por cada relación entre usuarios
- Dashboard de 'lo que me deben' vs 'lo que debo' con resumen consolidado por usuario

## 🚗 GRUPO 3 — PLATAFORMA DE CARPOOLING UNIVERSITARIO

**PROBLEMA: Sara, Aranda, Animero**

Estudiantes con carro publican sus rutas, horarios y cupos disponibles. Otros estudiantes pueden unirse a viajes que coincidan con su trayecto, el sistema valida compatibilidad de ruta y disponibilidad de cupo, y lleva el historial completo de viajes realizados.

**RETOS TÉCNICOS:**

- Matching de rutas por puntos de origen y destino compatibles (no tienen que ser idénticos)
- Gestión de cupos en tiempo real y manejo de cancelaciones con notificación al conductor
- Sistema de calificación acumulada que construye la reputación del perfil (pasajero y conductor)
- Estados del viaje: publicado → confirmado → en curso → completado → cancelado

## 📋 GRUPO 4 — CONTROL DE ASISTENCIA CON JUSTIFICACIONES

**PROBLEMA: Samuel, Daniel , Farid**

Sistema para cualquier organización — empresa, institución, grupo comunitario — que registra sesiones periódicas, permite marcar asistencia por miembro, acepta justificaciones con soporte documental cuando alguien falta, y genera reportes detallados por persona y por período.

**RETOS TÉCNICOS:**

- Roles diferenciados: administrador que crea sesiones vs. miembro que justifica su inasistencia
- Flujo de aprobación de justificaciones: pendiente → aprobada → rechazada con comentario del admin
- Reportes de asistencia por persona, por sesión y por período de tiempo
- Cálculo automático de porcentaje de asistencia con alertas al bajar del umbral configurado

## 💰 GRUPO 5 — SIMULADOR DE CRÉDITOS CON ANÁLISIS DE RIESGO

**PROBLEMA: Cruz, Quipo, Marin**

Sistema donde una persona solicita un crédito, ingresa sus datos financieros (ingresos mensuales, deudas actuales, historial de pagos), y el sistema evalúa su perfil de riesgo para aprobar, condicionar o rechazar el préstamo con una justificación detallada y el comparativo de escenarios.

**RETOS TÉCNICOS:**

- Fórmulas financieras reales: cuota fija mensual, tasa efectiva mensual, total a pagar e intereses
- Motor de reglas de negocio para scoring de riesgo (lógica condicional bien estructurada, no IA)
- Diferentes productos de crédito con condiciones distintas: libre inversión, vehículo, estudio
- Historial de solicitudes por usuario con comparativo de escenarios bajo distintas tasas y plazos

## 🎮 GRUPO 6 — SISTEMA DE TORNEOS AUTOMATIZADOS

**PROBLEMA: Christipher, Juan Angel, Salazar**

Plataforma para gestionar torneos de cualquier disciplina — fútbol, videojuegos, ajedrez — desde las inscripciones hasta la final, con generación automática de llaves (brackets) y avance de rondas según los resultados registrados.

**RETOS TÉCNICOS:**

- Algoritmo de generación de brackets: eliminación simple con manejo de byes para potencias de 2
- Automatización del avance: al registrar el resultado de un enfrentamiento, el ganador pasa solo a la siguiente fase
- Estados del torneo: inscripciones abiertas → en curso → ronda X → finalizado con campeón
- Panel público de resultados (solo lectura) vs. panel de administración (escritura)

## 🏥 GRUPO 7 — SISTEMA DE TRIAGE Y PRIORIZACIÓN DE ATENCIÓN

**PROBLEMA: forero, Castañeda y guzman**

Sistema para centros de salud o urgencias que clasifica a los pacientes según su nivel de gravedad al ingresar, los ubica en una cola dinámica de atención, y redistribuye prioridades si el estado de un paciente cambia mientras espera. Los médicos y enfermeros pueden ver y gestionar el flujo en tiempo real.

**RETOS TÉCNICOS:**

- Cola de prioridad dinámica: no es FIFO sino por nivel de urgencia combinado con tiempo de espera acumulado
- Reclasificación en tiempo real: si el estado de un paciente empeora, sube automáticamente en la cola
- Asignación a recursos disponibles: consultorios y médicos de turno con control de disponibilidad
- Dashboard en tiempo real del estado de la sala: pacientes esperando, en atención, atendidos por turno

## 🗺️ GRUPO 8 — SISTEMA DE RUTAS DE RECOLECCIÓN OPTIMIZADAS

**PROBLEMA: Ledys, Romero**

Una empresa de recolección (domicilios, correspondencia, residuos) necesita planificar las rutas diarias de sus operarios según zonas asignadas, puntos de parada registrados y capacidad del vehículo. El sistema asigna rutas, registra el avance en tiempo real y detecta puntos que quedaron sin atender para reagendarlos.

**RETOS TÉCNICOS:**

- Asignación de puntos de parada a rutas según zona geográfica y capacidad máxima del vehículo
- Seguimiento del avance de la ruta: cada punto tiene estado pendiente → visitado → omitido con razón
- Reagendamiento automático de puntos omitidos al siguiente ciclo de recolección
- Reporte de cobertura por zona y por operario: puntos atendidos vs. omitidos con motivo

## 📄 GRUPO 9 — FACTURACIÓN CON FLUJO DE APROBACIÓN EMPRESARIAL

**PROBLEMA: Nicolas, Jorlin, Sanchez**

En una empresa, cualquier empleado puede generar una solicitud de compra o gasto. Esa solicitud recorre una cadena de aprobación definida según el monto — jefe inmediato, luego gerencia, luego dirección financiera — y solo al final se genera la orden de compra o el rechazo formal con observaciones.

**RETOS TÉCNICOS:**

- Cadena de aprobación dinámica: los niveles que debe recorrer la solicitud dependen del monto y están configurados en el sistema
- Cada aprobador solo ve las solicitudes que están pendientes en su nivel específico
- Notificación al solicitante en cada cambio de estado con el comentario del aprobador actual
- Trazabilidad completa del recorrido de cada solicitud: quién aprobó, cuándo, qué observó

## 🧠 GRUPO 10 — PLATAFORMA DE EVALUACIONES CON BANCO DE PREGUNTAS ADAPTATIVO

**PROBLEMA: andres, rubio, katherine**

Un instructor o empresa crea un banco de preguntas organizadas por temas y nivel de dificultad. Cuando un usuario inicia una evaluación, el sistema selecciona preguntas según su desempeño en tiempo real: si va respondiendo bien sube la dificultad, si va mal baja. Al final genera un reporte de fortalezas y debilidades por tema.

**RETOS TÉCNICOS:**

- Motor de selección adaptativa: la siguiente pregunta se elige según el resultado de las anteriores, no de forma aleatoria pura
- Banco de preguntas con categorías, nivel de dificultad (básico / intermedio / avanzado) y peso configurable
- Evaluación con límite de tiempo por pregunta y por sesión completa con cuenta regresiva visible
- Reporte final por evaluado con mapa de competencias por tema: porcentaje de acierto por categoría y nivel

## 6. DOCUMENTACIÓN Y SUSTENTACIÓN

La documentación forma parte integral de la nota. Debe entregarse como un documento PDF o Word junto con el enlace al repositorio. Debe incluir obligatoriamente:

- Portada con nombres completos, rol de cada integrante, nombre del proyecto y fecha
- Descripción general del sistema: qué hace, para quién y qué problema resuelve
- Diagrama Entidad-Relación (ER) de la base de datos
- Diagrama de arquitectura del sistema (frontend, backend, base de datos, despliegue)
- Capturas del proceso de desarrollo de cada integrante: no solo pantallas del resultado final, sino del proceso (código en construcción, errores resueltos, avance por sesión)
- Instrucciones de instalación local y enlace al sistema desplegado
- Enlace al repositorio con evidencia del historial de commits
- Dificultades encontradas y cómo las resolvieron (mínimo media página)

## 7. DINÁMICA DE PRESENTACIÓN — FERIA DE PROYECTOS

La sustentación se realizará en una sesión separada a la entrega, en formato de feria de proyectos (stand). Cada grupo prepara una estación con su sistema funcionando y lo presenta a un grupo evaluador asignado al azar.

| Fase | Descripción |
|---|---|
| **Preparación (15 min)** | Cada grupo organiza su estación: computador con el sistema abierto, documentación impresa o en pantalla, y asigna quién explica qué parte según su rol. |
| **Ronda de evaluación (20 min por stand)** | El grupo evaluador asignado visita el stand. Los presentadores demuestran el sistema en funcionamiento y explican sus decisiones técnicas. El grupo evaluador hace al menos 3 preguntas técnicas y diligencia la rúbrica de pares. |
| **Cierre (10 min)** | El instructor realiza un recorrido final por todos los stands, valida el funcionamiento y hace preguntas individuales a cada integrante según su rol para la nota del instructor. |

La asignación de qué grupo evalúa a cuál se realizará por sorteo público el mismo día de la feria. Ningún grupo sabe con anticipación a quién evaluará, lo que incentiva que todos preparen buenas preguntas.

## 8. RÚBRICA DE EVALUACIÓN

La nota final se compone de dos partes: 60% evaluación del instructor y 40% evaluación del grupo par. Ambas se califican sobre 5.0.

### 8.1 Rúbrica del instructor (60%)

| Criterio | Descripción | Peso | Nota /5.0 |
|---|---|---|---|
| Código funcional | El sistema corre, cumple los requisitos del problema y maneja errores básicos | 35% | |
| Diseño de base de datos | Relaciones correctas, normalización mínima 3FN, constraints e integridad referencial | 20% | |
| Documentación + sustentación | Doc técnica completa con capturas del proceso individual de cada integrante, diagrama ER, instrucciones de despliegue | 25% | |
| Manejo de Git | Mínimo 10 commits por integrante con mensajes descriptivos, uso de ramas, al menos 1 Pull Request documentado | 20% | |
| **TOTAL INSTRUCTOR (60% de la nota final)** | | **100%** | |

## 8.2 Rúbrica de evaluación entre pares (40%)

| Criterio | Indicadores a evaluar | Nota /5.0 |
|---|---|---|
| Claridad de la presentación | ¿Se entiende qué hace el sistema? ¿Lo explicaron con claridad en el stand? | |
| Demostración funcional | ¿El sistema corrió durante la presentación? ¿Mostraron flujos completos? | |
| Manejo de preguntas | ¿Supieron responder las preguntas del grupo evaluador sobre su código y decisiones? | |
| Complejidad y esfuerzo percibido | ¿El problema fue bien abordado? ¿Se nota trabajo real más allá de lo básico? | |
| **TOTAL PARES (40% de la nota final)** | Promedio de los 4 criterios anteriores | |

Fórmula de la nota final: Nota Final = (Nota Instructor × 0.60) + (Nota Pares × 0.40)

# 9. FECHAS CLAVE

| Hito | Fecha | Entregable |
|---|---|---|
| Conformación de grupos y elección de stack | _____ | README inicial en el repositorio |
| Entrega del proyecto | _____ | Repositorio + documentación + enlace al sistema desplegado |
| Feria de proyectos (sustentación) | _____ | Sistema funcionando en el stand + rúbrica de pares diligenciada |

## 10. FIRMAS

| Instructor responsable: | Aprendiz (representante de grupo): |
|---|---|
| _____ Nombre: _____ Firma: _____ Fecha: _____ | _____ Nombre: _____ Firma: _____ Fecha: _____ |